CLASE 2 · TRACK DE INTELIGENCIA ARTIFICIAL

LangChain

Una capa de abstracción sobre los modelos de lenguaje



Duoc UC · Viña del Mar

CLASE 2

Índice

1. **El problema** — Por qué el código de la clase 1 no escala
2. **¿Qué es LangChain?** — Framework, no modelo ni proveedor
3. **Ventajas** — Abstracción, herramientas, cadenas y memoria
4. **Tipos de mensaje** — System, Human y AI Message
5. **Cuándo NO usarlo** — El costo de abstraer

PUNTO 1

El problema

Todo funcionó en la clase 1… hasta que el proyecto crece.

Con el SDK a secas

- Cada proveedor trae su propio cliente y su propia sintaxis
- Cambiar de modelo significa reescribir código
- El historial se arma a mano, mensaje por mensaje
- La misma lógica se repite en cada script

Lo que necesitamos

- Una sola interfaz para hablar con cualquier modelo
- Cambiar de proveedor tocando una línea
- Piezas reutilizables en vez de código copiado
- Concentrarnos en la lógica, no en el código repetitivo

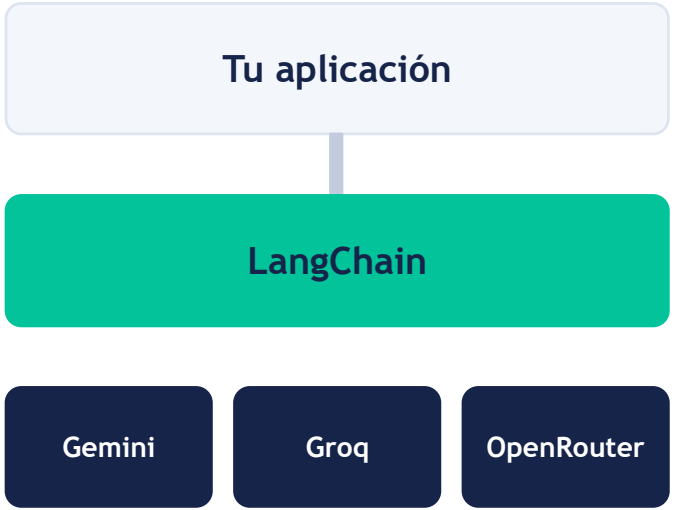
PUNTO 2

¿Qué es LangChain?

Un framework que simplifica el desarrollo de aplicaciones con modelos de lenguaje.

No es un modelo. No es un proveedor.

Es una capa de código que se sienta entre tu aplicación y los modelos: tú le hablas a LangChain, y LangChain se entiende con Gemini, Groq u OpenRouter.



Un mismo código, distintos motores detrás.

PUNTO 2

¿Y en una aplicación real?

LangChain no es la aplicación: es una capa dentro del backend.

1. El frontend envía el mensaje del usuario al backend.
2. El backend LEE el historial de la base de datos.
3. Arma el prompt: historial + mensaje nuevo.
4. Llama al modelo a través de LangChain.
5. GUARDA el mensaje y la respuesta en la BD.
6. Devuelve la respuesta al frontend.

La base de datos se usa dos veces: se lee antes de llamar al modelo y se escribe después.

El modelo no recuerda nada entre llamadas: cada petición le manda la conversación completa.

PUNTO 3

Ventajas · 1 de 2

1 Abstracción

Cambiar de proveedor sin reescribir tu código.

Sin LangChain: escribes un cliente distinto para cada proveedor.

Con LangChain: cambias la clase del modelo y el resto sigue igual.

2 Herramientas

Código que igual escribirías, ya resuelto.

Sin LangChain: armas los prompts concatenando texto y extraes la respuesta a mano.

Con LangChain: usas plantillas y parsers ya hechos y probados.

Las dos primeras ventajas atacan el problema de la diapositiva anterior.

PUNTO 3

Ventajas · 2 de 2

3 Cadenas

Definir la secuencia una vez y reutilizarla.

Sin LangChain: repites los mismos pasos escritos a mano en cada llamada.

Con LangChain: los defines en una línea: prompt | modelo | parser.

4 Memoria

Insertar el historial sin rearmar el prompt cada vez.

Sin LangChain: concatenas los mensajes previos a mano en cada llamada.

Con LangChain: le pasas la lista y él la coloca en su lugar.

Las cuatro ventajas apuntan a lo mismo: escribir menos código repetitivo y más lógica.

PUNTO 4

Tipos de mensaje

LangChain reemplaza los diccionarios de la clase 1 por objetos con nombre propio.

S

SystemMessage

Define el comportamiento y las reglas del modelo.

`role: "system"`

H

HumanMessage

Lo que escribe la persona usuaria.

`role: "user"`

AI

AIMessage

La respuesta del modelo, que se guarda como contexto.

`role: "assistant"`

El contenido es el mismo que ya vieron; cambia la forma de escribirlo.

PUNTO 5

Cuándo NO usarlo

Una sola llamada	Con una sola respuesta, el SDK del proveedor es más directo.
Necesitas control fino	La abstracción esconde parámetros del request.
Depuración	Cada capa extra hace más difícil ubicar dónde está el error.
Dependencia externa	La librería cambia rápido: lo de hoy puede no servir mañana.

Abstraer tiene un costo, y se paga en control y en transparencia.

AI notebook

2-langchain_model_api.ipynb